

1. Interfaces gráficas en Java. Contenedores.

Las interfaces gráficas de usuario, también conocidas como GUI, son aplicaciones software que permiten al usuario interactuar con ellas mediante elementos gráficos para representar información y reaccionar a las interacciones del usuario.

El propósito por el que se desarrollan las interfaces gráficas es para dar acceso a los recursos del sistema a usuarios sin necesidad de conocimientos específicos del mismo. Los sistemas operativos modernos, la gran mayoría de aplicaciones o todo el ecosistema web están basados en interacción mediante interfaces gráficas.

En esta unidad de trabajo no profundizaremos sobre principios de diseño o detalles de bajo nivel sobre el funcionamiento de las interfaces, ya que quedan fuera del alcance de este curso. En la unidad de trabajo entenderemos el funcionamiento de algunas de las interfaces gráficas de Java y haremos ejemplos sencillos de programas con interfaz gráfica, entendiendo su filosofía de funcionamiento.

Librerías Java para el desarrollo de interfaces gráficas

En la actualidad Java no es uno de los lenguajes más empleados para desarrollar interfaces de usuario, pero dada su evolución histórica el lenguaje cuenta con algunas librerías de desarrollo de interfaces. Entendiendo cómo funcionan estas interfaces, algunos principios de diseño y la filosofía tras los componentes y eventos, podremos aprender a desarrollar interfaces en otros lenguajes de programación o con otras tecnologías de forma sencilla gracias al conocimiento aprendido en esta unidad de trabajo.

Java cuenta con tres librerías para el desarrollo de interfaces de usuario:

- AWT: primera librería gráfica, que empleaba los componentes gráficos del propio sistema operativo en el que se ejecuta.
- Swing: la segunda librería gráfica fue Swing, que implementa nativamente en Java los componentes y traía algunas mejoras sobre AWT. Cuando se desarrollan interfaces gráficas con Swing se siguen empleando algunas clases de AWT, puesto que siguen siendo válidas.

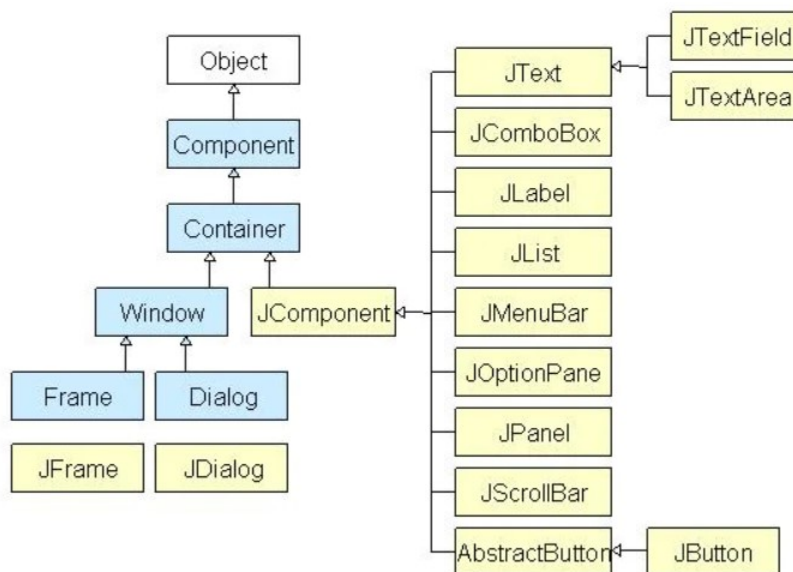
- JavaFX: con la evolución de las interfaces gráficas debido a los dispositivos móviles y la web, las herramientas proporcionadas por Swing eran insuficientes y se desarrolló JavaFX para aportar nuevas funciones y un desarrollo de interfaces más fácil de adaptar y de aplicar estilos, similar a otras herramientas modernas de diseño de interfaces.

En esta unidad de trabajo emplearemos Swing y AWT, que nos aportan una aproximación sencilla al desarrollo de interfaces gráficas y sigue teniendo un gran valor didáctico.

Java Swing

Java Swing nos aporta un conjunto de componentes que podremos usar para desarrollar interfaces gráficas, estos componentes pueden ser etiquetas de texto, cajas para introducir texto por parte del usuario, botones, listas, etc.

En la siguiente imagen puedes ver un resumen de la jerarquía de componentes de Swing:



JFrame

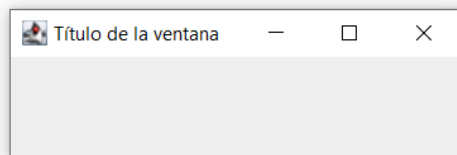
El componente principal sobre el que se va a crear la interfaz gráfica en Swing es de la clase JFrame (consulta <https://docs.oracle.com/javase/tutorial/uiswing/index.html>)

JFrame crea una ventana en nuestro sistema gráfico de ventanas de nuestro sistema operativo. JFrame es solo el contenedor de la ventana, pero debemos añadir paneles u otros elementos para generar la interfaz de usuario.

Otra clase de tipo ventana son los JDialog, que veremos en próximas lecciones de esta unidad de trabajo.

Así podemos crear un JFrame: `JFrame miFrame = new JFrame("Título de la ventana");`

Solo con un JFrame tendremos algo así:



JPanel

JPanel es un contenedor que puede agrupar un conjunto de componentes. Este tipo de componente debe estar contenido dentro de un JFrame.

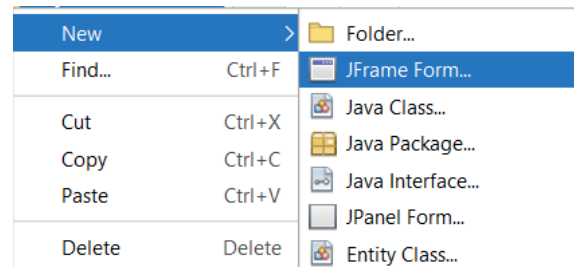
Así podemos crear un JPanel y añadirlo al JFrame:

```
JPanel panel1 = new JPanel();  
miFrame.add(panel1);
```

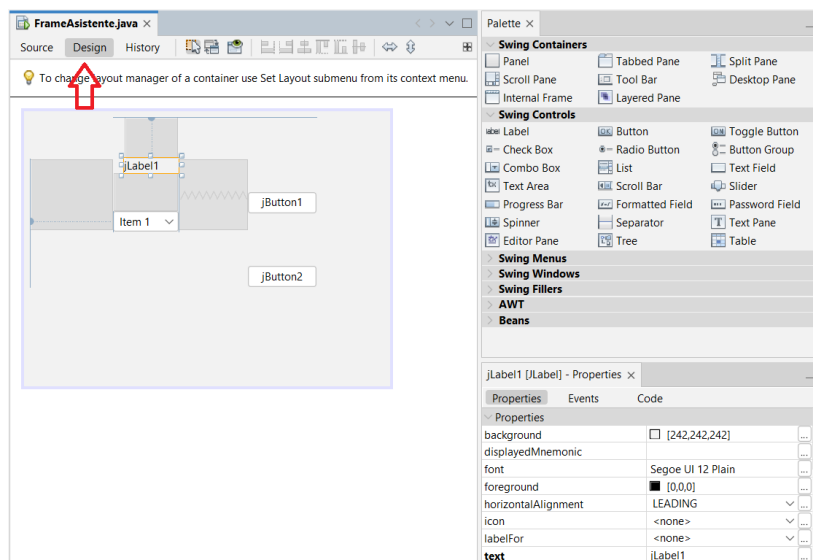
Asistentes de diseño de interfaces

Algunos IDE de desarrollo de software ofrecen herramientas de asistencia y creación de código automático para ayudar con el desarrollo de interfaces gráficas. Al igual que podemos generar constructores, métodos getters y setters, equals y hashCode de forma automática, las herramientas de diseño gráfico de interfaces de usuario alivian la tarea de creación del código de interfaces. Es importante entender que estas herramientas ayudan con la generación, pero no generan software que cumplan con los requisitos de nuestra aplicación; su labor es crear el código más monótono de forma rápida para que podamos adentrarnos en desarrollar las partes más relevantes y relacionadas con los requisitos propios de nuestra aplicación.

En NetBeans, si generamos una clase gráfica, como JFrame o JPanel:



El IDE nos permitirá ver su código fuente, como con todas las demás clases, pero también ofrecerá un modo “Design” en el que podremos crear algunas partes de nuestra interfaz gráfica mediante un asistente, en el que podremos arrastrar los componentes y configurar sus propiedades de forma sencilla. Es importante entender el código que generan estas herramientas automatizadas, si tras hacer las configuraciones pinchamos en el modo “Source”, podremos ver todo el código fuente que ha creado NetBeans con nuestra configuración en el modo “Design”.



Ejercicios

Ejercicio 1.

Crea una clase de tipo JFrame con NetBeans. Empleando el diseñador de interfaces prueba diferentes componentes, modifica sus propiedades y parámetros. Finalmente lee el código generado por la herramienta e intenta entender la relación entre lo que has hecho de forma gráfica y el código generado.

Ejercicio 2.

Prueba a modificar las propiedades de “panel1” en el código adjunto con esta lección. Familiarízate con algunos de sus métodos.